1. Part 1: A Simple Unix Shell

1.1 List of Files- Part 1

List of Files are below:

File	Purpose
main.c	Entry point and main event loop of the shell. Initializes signal handlers, allocates command buffer on heap, displays prompt, reads user input via terminal module, expands history references, adds commands to history, splits commands on ; and & operators, and executes each command. Orchestrates the entire shell operation.
shell.h	Shared header defining all constants and core data structures used throughout the shell. Defines MAX_ARGS (1000), MAX_LINE (4096), MAX_COMMANDS (100), MAX_PIPELINE (20), PROMPT_MAX (100). Contains the Command structure that holds parsed command information including arguments, redirections, background flag, and execution operators.
parser.c	Command parsing module implementing character-by-character state machine parser. Handles single-quoted strings (literal), double-quoted strings (with escape sequences), backslash escaping, whitespace tokenization, redirection operators (<, >, >>, 2>, 2>>), environment variable expansion ($VAR), and command splitting on ; and & operators. Contains parse_command(), split_commands(), split_pipeline(), and parse_redirections() functions.
parser.h	Header file declaring parser module functions: parse_command(), split_commands(), has_pipeline(), split_pipeline(), parse_redirections(), cleanup_argv(), and print_command().
builtin.c	Implements built-in commands that execute directly in shell process without forking. Handles pwd (print working directory), cd (change directory with HOME support), prompt (change shell prompt), exit (terminate shell), and history (display command history). execute_builtin() checks if command is built-in and executes it if so. Declares global shell_prompt variable.
builtin.h	Header declaring execute_builtin() function and extern shell_prompt variable.
execute.c	Command execution module handling external command execution. Implements expand_wildcards() using glob() for * and ? patterns, setup_redirections() using dup2() for <, >, >>, 2>, 2>> redirection, execute_single_command() using fork/execvp for single commands, and execute_pipeline() for connecting multiple commands with pipes. Manages process creation, file descriptor manipulation, and process waiting.
execute.h	Header declaring execute_command(), execute_single_command(), and execute_pipeline() functions.
signals.c	Signal handling module. Implements sigchld_handler() that reaps terminated child processes using waitpid(WNOHANG). Implements setup_shell_signals() that installs SIGCHLD handler with SA_RESTART and SA_NOCLDSTOP flags and makes shell ignore SIGINT/SIGQUIT/SIGTSTP. Implements restore_default_signals() called in child processes to enable normal signal behavior.
signals.h	Header declaring setup_shell_signals(), restore_default_signals(), and reap_zombies() functions.

terminal.c	Raw terminal mode input handling module. Implements enable_raw_mode() using tcsetattr() to disable line buffering, echo, and signal generation. Implements read_command_with_history() that reads keystrokes one at a time, handles arrow keys for history navigation, handles left/right arrows for cursor movement, handles backspace for deletion, and calls redraw_line() to display command with correct cursor position. Supports multi-line command editing when command exceeds terminal width.
terminal.h	Header declaring read_command_with_history() function for reading user input with history and editing support.
history.c	Command history management module maintaining circular buffer of last 100 commands. Implements add_history() to store commands, get_history_command() for numbered retrieval, get_history_by_prefix() for prefix searching, expand_history_reference() for !! and !n and !prefix expansion, print_history() to display history list, and cursor navigation functions for arrow key support.
history.h	Header declaring all history management functions including add_history(), get_history_command(), get_history_by_prefix(), print_history(), expand_history_reference(), get_last_command(), get_prev_history(), get_next_history(), reset_history_index(), and get_history_count().
Makefile	Build configuration file that compiles all source files (main.c, parser.c, builtin.c, execute.c, signals.c, terminal.c, history.c) with gcc -Wall -Wextra flags to produce shell executable. Provides clean target to remove compiled files and rebuild target.

1.2 Self-Diagnosis and Evaluation

Features Fully Implemented

Command Parsing and Tokenization

The shell correctly parses command lines that are a4096 characters long, and can handle up to 1000 arguments per command. Single-quoted strings are parsed in a way that all content between quotes is treated literally - no escape sequences or variable expansion occurs inside single quotes. Double-quoted strings allow escape sequences that include backslash-newline, backslash-tab, backslash-backslash, and backslash-quote. Variable expansion with $ works properly, replacing $HOME or other environment variables with their values. The parser recognizes and correctly handles multiple consecutive spaces and tabs as a single argument separator. Backslash escaping works both inside and outside quoted strings, allowing special characters to be included literally in arguments.

Command Execution

The shell successfully executes external commands by using the standard Unix fork/execvp mechanism. Commands can be specified by absolute path like /bin/ls, by relative path like

./show, or by bare command name, which is resolved using the PATH environment variable. The com_pathname field is correctly set to the first argument of the command.

Built-in Commands

The pwd command correctly prints the current working directory using getcwd(). The cd command changes the directory using chdir(). Additionally, it supports relative paths including parent directory navigation with .., and when called with no arguments returns to the HOME directory as specified in the environment. The cd command properly reports errors when trying to change to non-existent directories, without crashing the shell. The prompt command allows users to change the shell prompt string, with the new prompt persisting until it is changed again. The exit command terminates the shell process, with optional exit status codes. The history command displays a numbered list of all stored commands.

Wildcard Expansion

The shell expands wildcard patterns using the glob() standard library function. The asterisk * matches any sequence of characters which includes the empty sequence. The question mark ? matches exactly one character. When a wildcard pattern matches no files, the pattern is passed through literally to the command, which then produces an appropriate error message - this matches standard Unix shell behavior where typing ls *.nonexistent results in ls reporting that the file cannot be accessed. Wildcard expansion correctly handles hidden files and other filesystem edge cases.

Sequential and Background Execution

Commands separated by semicolons execute sequentially - the shell waits for each command to complete before starting the next. Commands followed by the ampersand & run in the background - the shell prints the process ID and returns to the prompt immediately without waiting for the background command to finish. Multiple commands can be combined on one line, for example: ls & pwd ; echo done runs ls in the background, then waits for pwd to finish, then runs echo done.

Input and Output Redirection

The less-than symbol < redirects standard input from a file. The greater-than symbol > redirects standard **Output** to a file, creating the file if it doesn't exist and truncating it if it does. The >> operator appends **Output** to a file instead of truncating. The 2> operator redirects standard error to a file, and 2>> appends error **Output**. Redirection operators work correctly with pipes - for example, cat file | grep pattern > results.txt correctly pipes the **Output** through grep and

then to a file. Multiple redirections can appear in the same command line and they interact correctly.

Pipelines

The pipe operator | connects commands so the standard **Output** of one becomes the standard input of the next. The shell correctly handles multiple pipes in a single command - typing cat /etc/passwd | cut -d: -f1 | sort | head -5. And, it correctly chains four commands together with data flowing through them. The shell creates the necessary pipes, forks a child process for each stage, connects the pipes, and waits for all stages to complete.

Command History

The shell stores the last 100 commands executed. The up arrow key recalls previous commands, moving backward through history. The down arrow key moves forward through history or returns to an empty line if at the end. The !! operator re-executes the last command. The !n operator re-executes command number n where n is shown in the history list. The !prefix operator re-executes the most recent command that starts with the given prefix. When history expansion occurs, the shell displays the expanded command text before executing it, and the expanded text is added to history (not the !... shorthand).

Terminal Input and Editing

The shell switches the terminal to raw mode so input is available character-by-character , rather than line-by-line. As you type, characters appear on the screen correctly. The left and right arrow keys move the cursor within the current command line. Backspace deletes the character before the cursor. When you move backward through history using the up arrow, the previous command is displayed in the editing buffer and you can move the cursor within it. The shell correctly handles multi-line commands that exceed the terminal width, calculating where the cursor is and redrawing appropriately.

Signal Handling

The shell ignores the SIGINT signal generated by pressing Ctrl-C buttons, so pressing Ctrl-C does not terminate the shell itself. When a foreground command is running and you press Ctrl-C, the signal is delivered to that command process instead, which causes the command to terminate while the shell continues. Similarly, the shell ignores SIGQUIT (Ctrl-) and SIGTSTP (Ctrl-Z). When a child process exits, the kernel automatically runs the shell's SIGCHLD handler, which calls waitpid() to clean up the process table entry. This prevents zombie

processes from accumulating. The handler uses WNOHANG to check for all exited children without blocking, and NOCLDSTOP so stopped processes don't trigger the handler.

Error Handling

When you type a command that doesn't exist, the shell prints "command not found", and returns to the prompt without crashing. When you type invalid syntax like unclosed quotes or invalid redirection, the parser gracefully handles it. When you press Enter on an empty line, the shell shows the prompt again. When a background command completes, its exit is handled automatically by the SIGCHLD handler without disrupting the current prompt or command editing.

Compilation

The Makefile compiles all source files with gcc -Wall -Wextra flags that enable warnings about questionable coding practices. The final compiled shell executable has zero warnings and compiles cleanly.

Features Not Fully Implemented or Missing

Memory Management in Glob Expansion

When expand_wildcards() calls glob() to expand patterns like *.c, the glob() function allocates memory internally. After copying the matched filenames and calling globfree() to free glob's internal structures, the individual filename strings are kept in the command's argv array for execution. These strings are never explicitly freed before the command executes. For short-running commands this is negligible, but in a long-running shell process with heavy wildcard usage, small memory allocations could accumulate. A production implementation would explicitly free these allocated strings after the child process is forked but before execvp(), or use a different memory management strategy.

Background Job Tracking

While the shell executes background commands correctly, there is no jobs builtin command that lists currently running background processes. Once a command is backgrounded with &, there is no way to bring it back to the foreground or send it to the background. Suspended processes (stopped with Ctrl-Z) are reported with a message but not tracked - you cannot use commands like fg or bg to manage them.

History Persistence

Command history is stored in memory in an array of 100 entries. When the shell exits, this history is lost. There is no ~/.history file or similar mechanism to persist history across shell sessions.

History Size Limit

The history array is fixed at 100 entries. For users who work with the shell for long sessions, older commands beyond the 100 most recent are discarded.

Limited Terminal Key Support

The shell only handles arrow keys (up, down, left, right) and backspace for editing. The Delete key, Home key, End key, Page Up, Page Down, and other extended keys are not recognized. Sequences for these keys may display as garbage characters when pressed.

No Environment Variable Expansion in Unquoted Context

The parser does implement $VAR expansion, but only for complete tokens. Partial variable expansion like echo $HO doesn't work - $HO is looked up in the environment and if not found, it remains as-is. Complex variable expansions like ${VAR:0:5} are not supported.

No Aliases

The shell does not support creating aliases like alias ll='ls -la'. Each command name must be the actual command.

No Script Mode

The shell is interactive only. It doesn't support reading commands from a file as a script file argument.

No Redirection Append for Stdin

There's no << operator for "here documents" that let you provide multi-line input inline.

3.3. Discussion Of Solution

Design Approach and Architecture

The shell is arranged into separate modules, each responsible for a distinct aspect of functionality. The main loop in main.c manages the entire process. But main loop delegates actual work to specialized modules. The modular design makes the code easier to comprehend, test, and maintain. Each module can be tested independently without running the entire shell.

The Command structure serves as the central data structure that flows through the system. The parser populates Command structures with arguments, redirections, and execution flags. The builtin module checks if a Command is a built-in. The execute module handles the actual execution. The signals module provides the execution environment. This separation of concerns means that modifications to one aspect means one does not need to understand all of the code.

Technical Choices Made

Why a Character-by-Character State Machine Parser Instead of strtok()

The parser implements a state machine that tracks whether it's inside single quotes, double quotes, or after a backslash. This approach is more complex than simply using the strtok() function from the C standard library, which splits strings on delimiter characters.

However, strtok() has a fundamental limitation for shell parsing: it treats all delimiters identically. When you call strtok(input, " "), it splits on every space, regardless of whether that space is inside quotes. The shell needs to distinguish between a space that separates arguments and a space that's part of a quoted string.

The state machine approach correctly handles this. When parsing "hello world", the state machine sees the opening quote, enters single-quote mode, collects all characters including the space, sees the closing quote, and exits single-quote mode. The result is one token: hello world.

Additionally, the state machine needed to handle escape sequences. Inside double quotes, a backslash followed by certain characters has special meaning: \n becomes a newline, " becomes a quote character. The strtok() approach provides no mechanism for this.

The trade-off was acceptable: the state machine implementation is roughly 150 lines of code versus 20 lines if using strtok(), but the correctness gained is substantial. Users familiar with Unix shells expect their shell to handle quoting and escaping the way bash does, and the state machine provides that compatibility.

Why glob() Instead of Manual Wildcard Expansion

Wildcard expansion could have been implemented from scratch by opening directories, reading entries, and pattern matching. This was rejected in favor of using glob() from the standard C library.

The glob() function is battle-tested and handles edge cases correctly. It understands filesystem permissions, symlinks, hidden files, and pattern syntax thoroughly. Implementing these correctly from scratch is non-trivial and error-prone. By delegating to glob(), the shell gains reliability without bloating the codebase.

One important behavior: when a pattern matches no files, glob() can fail. Rather than deleting the pattern or printing an error, the shell passes the pattern through literally to the command. This matches standard Unix behavior - when you type ls *.nonexistent, ls itself prints an error message. This behavior was deliberately chosen to give the external command a chance to handle the error meaningfully.

Why Using fork()/execvp() Instead of system()

The C standard library provides a system() function that executes a shell command. Using system() would have been simpler - just call system("ls -la") instead of forking and execing.

However, system() spawns an entire shell to interpret the command. This adds overhead and introduces complexity. More importantly, system() makes it hard to implement redirections and pipelines - you'd need to pass shell syntax strings and rely on the spawned shell to interpret them correctly.

The fork()/execvp() approach gives direct control over the process and file descriptors. When you want to redirect stdin to a file, you fork(), call dup2(fd, STDIN_FILENO) in the child, then execvp(). The redirection happens before the new process image loads, so the new process inherits the redirected file descriptors. The parent process has control over pipes, file descriptors, and process management directly.

Why Raw Terminal Mode

By default, the terminal operates in "cooked" or "canonical" mode. This means the kernel buffers input by lines - you type characters and they don't get delivered to the program until you press Enter. Additionally, the kernel echoes characters you type to the screen, and special characters like Ctrl-C generate signals that interrupt the program.

For an interactive shell with history and line editing, this mode is limiting. To support arrow keys for history navigation and in-line cursor movement, the shell needs individual keystrokes immediately, before the user presses Enter. Raw mode achieves this.

In raw mode:

The terminal delivers characters one at a time

The shell handles all **Output** to the screen

Special key sequences (arrow keys, function keys) come through as byte sequences that the shell can interpret.

The trade-off is that the shell must handle everything: echoing typed characters, erasing for backspace, redrawing when history is recalled. The redraw_line() function handles the complex case where a command exceeds the terminal width - it calculates cursor position in terms of rows and columns, moves the cursor, and redraws from the start of the prompt.

Raw mode is essential for providing a good interactive experience that users expect from modern shells.

Signal Handling Architecture

Three complementary mechanisms work together:

The shell ignores SIGINT, SIGQUIT, and SIGTSTP. When you press Ctrl-C, the kernel delivers the signal to all processes in the foreground process group, but the shell has told the kernel to ignore it. So the shell continues running.

Child processes restore default signal handlers before executing. This is done with restore_default_signals() called in the child right before execvp(). When a child is running and you press Ctrl-C, the signal is delivered to the child, which terminates because it's listening to SIGINT normally.

The SIGCHLD handler reaps child processes. This handler is installed with special flags: SA_RESTART means "if a system call like read() is interrupted by this signal, automatically retry it" instead of returning EINTR. This simplifies the main loop - it doesn't need to check for EINTR everywhere.

An alternative approach using sigprocmask() to block signals in the parent and unblock in the child was considered but rejected because it introduces race conditions and requires careful management. The three-part approach is simpler and clearer.

Strengths of the Implementation

Modular and Maintainable

Each module (parser, execute, builtin, signals, terminal, history) has a clear responsibility. If you need to add a new built-in command, you modify builtin.c. If you want to change how history works, you modify history.c. This separation makes the code easier to understand and reduces the risk that changes break other parts.

Unix Compliant Parsing

The parser behaves the way Unix shells do. Single quotes prevent interpretation of special characters. Double quotes allow escaping. Backslash escaping works both inside and outside quotes. Variable expansion happens. Whitespace handling matches expectations. Users familiar with bash find the parsing familiar.

Robust Error Handling

The shell doesn't crash when given invalid input. Typing an undefined command returns to the prompt with an error message. Pressing Ctrl-C doesn't kill the shell. Using an invalid option like cd /nonexistent/ reports an error but keeps the shell running. Files that can't be found for redirection are reported appropriately. This robustness comes from careful error checking in critical sections and signal handling that protects the shell.

Clean Process Management

Fork and execvp are used correctly. The parent reaps children promptly via SIGCHLD, preventing zombies. Process groups are handled correctly so Ctrl-C reaches the right processes. File descriptor management in pipelines closes unnecessary descriptors in children so EOF propagates correctly.

Complete Feature Set

The shell implements essentially all the features needed for interactive use: command execution, pipelines, redirection, wildcards, history, and built-ins. A user could accomplish real work with this shell.

Compiles Without Warnings

All code compiles cleanly with gcc -Wall -Wextra, which means no suspicious constructs or undefined behavior.

Weaknesses and Limitations

Memory Leaks in Wildcard Expansion

When glob() expands patterns, it allocates memory internally. The shell calls globfree() to free glob's structures but doesn't free the individual strings that were copied into argv. In a long-running shell, especially with scripts that expand many wildcard patterns, memory accumulates. A fix would be to explicitly free these strings after forking but before execvp() in the child, since the child is about to load a new process image anyway and its memory will be reclaimed.

No Job Control

Background jobs run without tracking. There's no jobs command to list running background processes, no fg to bring a job to foreground, no bg to resume a suspended job. Once backgrounded, a process is essentially forgotten except for automatic cleanup when it exits. Modern shells maintain a job table for this reason.

Limited Terminal Support

Only arrow keys and backspace work. Delete, Home, End, and other keys generate escape sequences that aren't recognized. This is a minor limitation but affects user experience. Fixing this would require expanding the escape sequence parser to handle more sequences.

History Limited to 100 Commands

The history buffer is fixed size. Users working interactively for long sessions will lose older commands. A circular buffer approach would work but the current approach is simpler.

No Batch Mode

The shell is interactive only. It can't be run as a script (like ./myscript.sh) where commands come from a file instead of the terminal.

History Not Persistent

History is stored in memory and lost when the shell exits. Typical shells save history to ~/.history or ~/.bash_history for persistence across sessions.

No Environment Variable Display

Typing just a variable name doesn't display its value. You need to pipe it through echo: echo $HOME rather than just $HOME.

Potential Improvements

Memory Management

Explicitly tracking and freeing allocated memory from glob() expansions would prevent accumulation. This could be done with a cleanup function called after fork but before execvp.

Job Control

Implementing a job table that tracks background and suspended processes would enable fg, bg, and jobs commands. This requires more complex process tracking and signal handling.

Extended Terminal Support

Expanding the escape sequence handler in terminal.c to recognize and handle Delete, Home, End, and other sequences would improve usability.

Command Line Editing Enhancements

Features like Ctrl-U to clear the line, Ctrl-A and Ctrl-E to jump to line ends, and incremental history search would make the shell more usable.

Builtin Command Expansion

Adding more built-in commands like ls, cat, grep, or at least some commonly used ones, would reduce the overhead of spawning processes for simple operations.

Script Mode

Supporting a -c flag to execute a command and exit, or running commands from a file, would make the shell usable in batch scenarios.

2. Conclusion

The implementation successfully delivers all required functionality for a POSIX-compliant interactive shell with client-server remote access. Part 1 provides a feature-complete local shell that handles complex command syntax, pipelines, history, and signal management correctly. Part 2 extends this with network access while maintaining feature parity.

The modular architecture and careful signal handling make the shell robust and reliable. Extensive testing demonstrates that the shell handles both normal usage and edge cases gracefully.